PRAKTIKUM

PERANCANGAN DAN PEMROGRAMAN WEB

MODUL 7

AJAX API



Oleh:

Arif Stand Pramudya 103122400001

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

LEMBAR SOAL PRAKTIKUM
MODUL 07: AJAX DAN KONSUMSI API

Nama: Arif Stand Pramudya NIM: 103122400001

Kelas: SE-08-02 Tanggal: 4 November 2026

Kerjakan dua soal proyek berikut. Kedua program secara bersama-sama mencakup seluruh materi Modul 07. Gunakan data contoh dan jalankan melalui browser.

Cakupan dan bobot

Proyek	Materi yang diterapkan	Bobot
Soal 1	Konsep dan alur AJAX; event handling; XMLHttpRequest; Fetch; API lokal/eksternal; loading dan error.	50 poin
Soal 2	Axios GET dan POST; loading/error; useState dan useEffect pada React.	50 poin

Persiapan dan cara menjalankan

1. Gunakan Paket_Praktikum_AJAX.zip dari modul sebelumnya. Ekstrak seluruh paket dan pastikan Node.js 18 atau lebih baru tersedia.
2. Di dalam paket, buat latihan-praktikan/soal-01 dan latihan-praktikan/soal-02. Simpan kode masing-masing proyek pada folder tersebut.
3. Jalankan JALANKAN.bat atau node server.cjs dari folder paket. Biarkan terminal berjalan.
4. Buka http://127.0.0.1:3007/latihan-praktikan/soal-01/ atau alamat soal-02/. Sesuaikan port jika server memakai port lain.

API dan data yang digunakan

Metode dan URL	Keterangan
GET /api/users	Array tiga pengguna: id, name, email.
GET /api/users?mode=empty	HTTP 200 dengan array kosong.
GET /api/users?mode=error	HTTP 503 untuk simulasi gagal.
POST /api/users	Kirim JSON name dan email; HTTP 201, id 101, simulated true. Data tidak disimpan.
GET https://jsonplaceholder.typicode.com/users	API eksternal; membutuhkan internet.

Tambahkan parameter delay dalam milidetik, maksimal 5000. Contoh: /api/users?delay=1500 atau /api/users?mode=error&delay=1000. Mock lokal dapat dipakai tanpa internet.

Ketentuan umum

Simpan data yang ditampilkan berdasarkan response. Tampilkan pesan yang mudah dipahami, gunakan label input, dan tangani kegagalan. Referensi kode boleh memakai contoh pada modul. Skenario uji dan ketentuan pengumpulan tersedia pada halaman 4.

Soal 01. Portal informasi praktikum

Materi: AJAX, event, XMLHttpRequest, Fetch, API lokal/eksternal | Bobot: 50 poin | Folder: soal-01

Laboratorium membutuhkan portal untuk membaca pengumuman dan daftar peserta tanpa memuat ulang seluruh halaman. Buat satu halaman HTML dan JavaScript dengan dua bagian yang saling melengkapi: pengumuman dan direktori peserta.

Berkas yang dibuat

Buat index.html, app.js, dan pengumuman.txt di folder soal-01. Isi pengumuman.txt sebagai berikut:

```
PENGUMUMAN PRAKTIKUM
Praktikum dimulai pukul 08.00.
Bawa kartu praktikum dan rapikan komputer setelah selesai.
```

Kebutuhan program

1. Sediakan kolom Catatan praktikan, tombol Tambah hitungan, angka awal 0, dan waktu halaman pertama dibuka. Catatan, hitungan, dan waktu awal harus tetap ada setelah request selesai.
2. Pada bagian pengumuman, sediakan tombol Muat pengumuman. Event klik harus meminta pengumuman.txt menggunakan XMLHttpRequest secara asynchronous. Periksa readyState dan status HTTP, lalu tampilkan responseText pada elemen pre.
3. Pada bagian direktori, sediakan pilihan sumber Mock lokal atau JSONPlaceholder dan tombol Ambil peserta. Gunakan Fetch untuk mengambil data dari URL sesuai pilihan.
4. Untuk mock, gunakan /api/users?delay=1500. Untuk sumber eksternal, gunakan URL JSONPlaceholder pada halaman 1. Periksa response.ok sebelum membaca JSON.
5. Tampilkan maksimal tiga pengguna pertama beserta emailnya, nama sumber, dan jumlah data yang ditampilkan. Gunakan data dari response; jangan mengisi nama peserta secara manual.
6. Saat menunggu, tampilkan pesan loading dan nonaktifkan tombol request yang sedang berjalan. Tombol Tambah hitungan tetap harus dapat digunakan. Aktifkan kembali tombol request setelah proses selesai.
7. Tampilkan pesan gagal untuk file tidak ditemukan, HTTP gagal, atau gangguan jaringan. Jika API eksternal gagal, arahkan pengguna untuk memeriksa internet atau memilih mock secara manual.

Hasil akhir yang diharapkan

Pengumuman dan direktori dapat diperbarui tanpa reload. Praktikan masih dapat mengetik catatan serta menambah hitungan ketika data sedang diminta. Hasil pengumuman berasal dari XHR, sedangkan daftar peserta berasal dari Fetch.

Penjelasan singkat dalam laporan

Jelaskan event pemicu, request yang dikirim, response yang diterima, dan elemen DOM yang berubah. Tunjukkan bukti bahwa interaksi hitungan tetap berjalan selama menunggu data.

Soal 02. Aplikasi registrasi peserta React

Materi: Axios GET/POST, loading/error, useState, useEffect | Bobot: 50 poin | Folder: soal-02

Buat satu aplikasi React untuk menampilkan daftar peserta dan mengirim formulir pendaftaran. Daftar dimuat otomatis, sedangkan data pendaftaran dikirim setelah pengguna menekan tombol.

Berkas dan library

Buat index.html dan app.jsx di folder soal-02. Siapkan elemen root pada body. Muat React, ReactDOM, Axios, dan Babel berurutan dari ../../vendor/, memakai nama file yang sama dengan contoh React pada modul. Muat app.jsx dengan tag berikut; pemasangan ReactDOM.createRoot() dilakukan di app.jsx.

```
<script type="text/babel" data-presets="react" src="app.jsx"></script>
```

Kebutuhan program

1. Gunakan useState untuk data peserta, loading, error, input formulir, dan status pengiriman. Gunakan useEffect untuk GET awal dan saat pilihan skenario berubah.
2. Ambil daftar memakai axios.get() dan baca response.data. Tampilkan nama serta email menggunakan key dari user.id. Sediakan tombol Muat ulang agar request dapat dijalankan lagi.
3. Sediakan pilihan Berhasil, Data kosong, Error HTTP, dan Timeout. Gunakan URL serta batas tunggu pada tabel di bawah. Tampilkan loading, hasil, kosong, atau pesan gagal sesuai kondisi sebenarnya.
4. Rapikan effect saat pilihan berubah atau komponen dilepas, misalnya dengan AbortController dan penanda effect aktif. Response lama tidak boleh menimpa hasil pilihan terbaru.
5. Sediakan form Nama, Email, dan tombol Kirim pendaftaran. Cegah reload dengan preventDefault(), tolak nama kosong atau hanya spasi, dan validasi format email.
6. Kirim objek name dan email memakai axios.post("/api/users", objekData). Nonaktifkan tombol saat mengirim. Tampilkan status HTTP, id, nama, email, simulated, serta pesan Simulasi berhasil, data tidak disimpan.
7. Tangani kegagalan GET dan POST, lalu pulihkan kontrol setelah selesai. Pendaftaran tidak menambah data ke server; tampilkan response POST terpisah dari daftar GET.

Skenario GET	URL	Timeout Axios
Berhasil	/api/users?delay=1000	5000 ms
Data kosong	/api/users?mode=empty&delay=1000	5000 ms
Error HTTP	/api/users?mode=error&delay=1000	5000 ms
Timeout	/api/users?delay=2500	700 ms

Gunakan try, catch, dan finally untuk mengelola kondisi proses. Batas waktu 700 ms harus menghasilkan kegagalan timeout sebelum response yang dijeda 2500 ms tiba.

Pengujian, pengumpulan, dan penilaian

Jalankan skenario berikut

Soal	Pengujian	Hasil yang harus terlihat
1	Muat pengumuman, lalu ubah isi file dan muat lagi.	Isi baru tampil melalui XHR.
1	Ubah sementara URL pengumuman menjadi hilang.txt.	Pesan gagal dengan status 404. Kembalikan URL setelah uji.
1	Ambil mock sambil mengetik dan klik hitungan tiga kali.	Tiga peserta tampil; catatan tetap ada, hitungan bertambah, waktu awal tetap sama.
1	Pilih API eksternal dan kirim request.	Maksimal tiga pengguna tampil atau pesan gagal yang jelas. Sumber tidak berganti otomatis.
2	Buka halaman, lalu klik Muat ulang.	GET berjalan otomatis saat awal; klik memicu request baru.
2	Coba Data kosong dan Error HTTP.	Pesan kosong atau HTTP 503; daftar lama dibersihkan.
2	Coba Timeout, lalu kembali ke Berhasil.	Request timeout pada batas 700 ms; aplikasi dapat digunakan lagi.
2	Ganti pilihan ketika request masih berjalan.	Hasil akhir mengikuti pilihan terbaru.
2	Kirim Rina Praktikan, rina@example.com.	HTTP 201; id 101; data sesuai input; simulated true. GET tetap tiga peserta.
2	Nama hanya spasi atau format email tidak valid.	Form menolak pengiriman dan memberi petunjuk perbaikan.

Jika internet atau layanan eksternal tidak tersedia, catat kondisi tersebut. Penanganan kegagalan menjadi bukti uji jalur eksternal. API lokal dan POST di atas tetap dapat diuji tanpa internet.

Berkas yang dikumpulkan

Kumpulkan NIM_Nama_Modul07.zip berisi folder soal-01, soal-02, README cara menjalankan pada paket awal, dan laporan hasil uji. Sertakan tangkapan layar kedua program, bukti POST, serta satu kondisi gagal. Catat hasil aktual setiap skenario dan jelaskan alur XHR, Fetch, Axios, serta peran state/effect React secara singkat.

Rubrik penilaian (nilai akhir maksimal 100 poin)

Aspek	Soal 1	Soal 2
Fungsi utama sesuai kebutuhan	30	30
Loading, validasi, dan penanganan kondisi	10	10
Keterbacaan kode dan bukti pengujian	10	10
Jumlah	50	50

LEMBAR JAWABAN PRAKTIKUM
MODUL 07: AJAX DAN KONSUMSI API

Soal 01. Portal informasi praktikum

Kode:

index.html

```
<!doctype html>
<html lang="id">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Soal 1 | Portal Informasi Praktikum</title>
  <link rel="icon" href="data:,">
  <link rel="stylesheet" href="../../assets/style.css">
  <style>
    textarea {
      display: block;
      width: 100%;
      min-height: 104px;
      margin: 6px 0 16px;
      padding: 10px 12px;
      border: 1px solid var(--control);
      border-radius: 6px;
      background: var(--surface);
      color: var(--text);
      font: inherit;
      resize: vertical;
    }
    pre.result { min-height: 100px; white-space: pre-wrap; overflow-wrap:
anywhere; }
    .inline-stat { display: flex; flex-wrap: wrap; align-items: center; gap:
10px 18px; margin: 12px 0 18px; }
    .inline-stat p { margin: 0; }
    .subsection { margin-top: 24px; padding-top: 20px; border-top: 1px solid
var(--rule); }
    .field-note { color: var(--muted); font-size: 14px; }
  </style>
</head>
<body>
  <a class="skip" href="#utama">Lewati navigasi</a>
  <aside class="sidebar">
    <a class="brand" href="../../index.html"><span class="brand-mark" aria-
hidden="true">{ }</span> Praktikum AJAX</a>
    <div class="scope">Modul 07 / Proyek praktikan</div>
    <nav aria-label="Navigasi proyek">
      <a href="../../index.html">Peta materi</a>
      <a href="../soal-01/" aria-current="page">Soal 1 · Portal informasi</a>
      <a href="../soal-02/">Soal 2 · Registrasi React</a>
```

```
    </nav>
    <p class="sidebar-footer">Latihan AJAX, Fetch API, XMLHttpRequest, dan
konsumsi API.</p>
  </aside>

  <header class="topbar">
    <span>Modul 07 <span class="context">/ Soal 1</span></span>
    <a href="../../Modul_07_AJAX_Sederhana.pdf">Baca modul PDF</a>
  </header>

  <div class="mobile-nav">
    <label class="skip" for="pindah-proyek">Pilih proyek</label>
    <select id="pindah-proyek" aria-label="Pindah proyek"
onchange="location.href=this.value">
      <option value="../soal-01/" selected>Soal 1 · Portal informasi</option>
      <option value="../soal-02/">Soal 2 · Registrasi React</option>
    </select>
  </div>

  <main id="utama" class="sheet">
    <div class="file-warning" data-file-warning hidden>
      <strong>Jalankan server terlebih dahulu.</strong><br>
      Ekstrak seluruh ZIP, jalankan <code>JALANKAN.bat</code>, lalu buka halaman
melalui
      <a href="http://127.0.0.1:3007">http://127.0.0.1:3007</a>. Jangan membuka
file melalui <code>file://</code>.
    </div>

    <div class="page-meta">Proyek 1 / Bobot 50 poin</div>
    <h1>Portal Informasi Praktikum</h1>
    <p class="intro">Muat pengumuman menggunakan XMLHttpRequest dan ambil
direktori peserta menggunakan Fetch API. Kedua proses berjalan tanpa memuat
ulang halaman.</p>

    <div class="workspace">
      <section class="practice" aria-label="Pengumuman dan aktivitas praktikan">
        <h2>Aktivitas praktikan</h2>
        <label for="catatan">Catatan praktikan</label>
        <textarea id="catatan" placeholder="Tulis catatan selama
praktikum..."></textarea>
        <div class="inline-stat">
          <button type="button" id="tambah-hitungan">Tambah hitungan</button>
          <p>Jumlah klik: <strong id="jumlah-hitungan" aria-
live="polite">0</strong></p>
        </div>
        <p class="field-note" id="waktu-awal">Waktu halaman dibuka: sedang
dicatat...</p>

        <div class="subsection">
          <h2>Pengumuman praktikum</h2>
          <p class="field-note">Sumber data: file lokal
<code>pengumuman.txt</code>, diminta menggunakan XMLHttpRequest
```

```
asynchronous.</p>
          <button type="button" id="muat-pengumuman">Muat pengumuman</button>
          <p id="status-pengumuman" class="result" role="status" aria-
live="polite">Klik tombol untuk mengambil pengumuman.</p>
          <pre id="isi-pengumuman" class="result" aria-label="Isi
pengumuman">Belum ada pengumuman yang dimuat.</pre>
        </div>
      </section>

      <section class="practice" aria-label="Direktori peserta">
        <h2>Direktori peserta</h2>
        <label for="sumber-api">Pilih sumber data</label>
        <select id="sumber-api">
          <option value="mock">Mock lokal</option>
          <option value="eksternal">JSONPlaceholder</option>
        </select>
        <p class="field-note">Mock lokal memakai jeda 1.500 ms. JSONPlaceholder
memerlukan koneksi internet.</p>
        <button type="button" id="ambil-peserta">Ambil peserta</button>
        <p id="status-peserta" class="result" role="status" aria-
live="polite">Pilih sumber, lalu ambil peserta.</p>
        <p id="info-peserta" class="field-note">Sumber: belum dipilih · Data
ditampilkan: 0</p>
        <ul id="daftar-peserta" class="users" aria-label="Daftar peserta"></ul>
      </section>
    </div>

    <section class="explanation" style="margin-top:24px">
      <h2>Alur program</h2>
      <ol>
        <li>Event klik pada tombol Muat pengumuman menjalankan XMLHttpRequest
secara asynchronous.</li>
        <li>Program memeriksa <code>readyState</code> dan status HTTP. Jika
status berhasil, isi <code>responseText</code> ditampilkan pada elemen
<code>pre</code>.</li>
        <li>Event klik pada tombol Ambil peserta menjalankan Fetch sesuai sumber
yang dipilih.</li>
        <li>Program memeriksa <code>response.ok</code>, membaca JSON, lalu
menampilkan maksimal tiga nama dan email dari response.</li>
        <li>Selama request berjalan, tombol request dinonaktifkan. Catatan dan
tombol hitungan tetap dapat digunakan.</li>
      </ol>
      <p>Data direktori berasal dari response API. Nama peserta tidak ditulis
secara manual pada kode tampilan.</p>
    </section>

    <div class="pager">
      <a href="../../index.html">&larr; Peta materi</a>
      <a href="../soal-02/">Lanjut ke Soal 2 &rarr;</a>
    </div>
  </main>
  <script src="../../assets/ui.js"></script>
```

```
  <script src="app.js"></script>
</body>
</html>
```

app.js

```
const catatanInput = document.querySelector("#catatan");
const tombolHitung = document.querySelector("#tambah-hitungan");
const jumlahHitungan = document.querySelector("#jumlah-hitungan");
const waktuAwal = document.querySelector("#waktu-awal");

const tombolPengumuman = document.querySelector("#muat-pengumuman");
const statusPengumuman = document.querySelector("#status-pengumuman");
const isiPengumuman = document.querySelector("#isi-pengumuman");

const sumberApi = document.querySelector("#sumber-api");
const tombolPeserta = document.querySelector("#ambil-peserta");
const statusPeserta = document.querySelector("#status-peserta");
const infoPeserta = document.querySelector("#info-peserta");
const daftarPeserta = document.querySelector("#daftar-peserta");

const waktuHalamanDibuka = new Date();
waktuAwal.textContent = `Waktu halaman dibuka:
${waktuHalamanDibuka.toLocaleString("id-ID", {
  dateStyle: "full",
  timeStyle: "medium"
})}`;

let hitungan = 0;
tombolHitung.addEventListener("click", () => {
  hitungan += 1;
  jumlahHitungan.textContent = String(hitungan);
});

tombolPengumuman.addEventListener("click", () => {
  const xhr = new XMLHttpRequest();
  tombolPengumuman.disabled = true;
  statusPengumuman.dataset.state = "";
  statusPengumuman.textContent = "Memuat pengumuman...";
  isiPengumuman.textContent = "";

  xhr.open("GET", "pengumuman.txt", true);

  xhr.onreadystatechange = () => {
    if (xhr.readyState !== XMLHttpRequest.DONE) return;

    if (xhr.status >= 200 && xhr.status < 300) {
      isiPengumuman.textContent = xhr.responseText;
      statusPengumuman.dataset.state = "success";
      statusPengumuman.textContent = "Pengumuman berhasil dimuat.";
    } else {
      statusPengumuman.dataset.state = "error";
      statusPengumuman.textContent = `Gagal memuat pengumuman. HTTP
```

```
${xhr.status}. Periksa file pengumuman.txt dan coba lagi.`;
      isiPengumuman.textContent = "Pengumuman tidak dapat ditampilkan.";
    }
    tombolPengumuman.disabled = false;
  };

  xhr.onerror = () => {
    statusPengumuman.dataset.state = "error";
    statusPengumuman.textContent = "Gangguan jaringan saat memuat pengumuman.
Periksa server lokal.";
    isiPengumuman.textContent = "Pengumuman tidak dapat ditampilkan.";
    tombolPengumuman.disabled = false;
  };

  xhr.send();
});

tombolPeserta.addEventListener("click", async () => {
  const pilihan = sumberApi.value;
  const konfigurasi = pilihan === "mock"
    ? { nama: "Mock lokal", url: "/api/users?delay=1500" }
    : { nama: "JSONPlaceholder", url:
"https://jsonplaceholder.typicode.com/users" };

  tombolPeserta.disabled = true;
  statusPeserta.dataset.state = "";
  statusPeserta.textContent = `Memuat data dari ${konfigurasi.nama}...`;
  infoPeserta.textContent = `Sumber: ${konfigurasi.nama} · Data ditampilkan: 0`;
  daftarPeserta.replaceChildren();

  try {
    const response = await fetch(konfigurasi.url);
    if (!response.ok) {
      throw new Error(`HTTP ${response.status}`);
    }

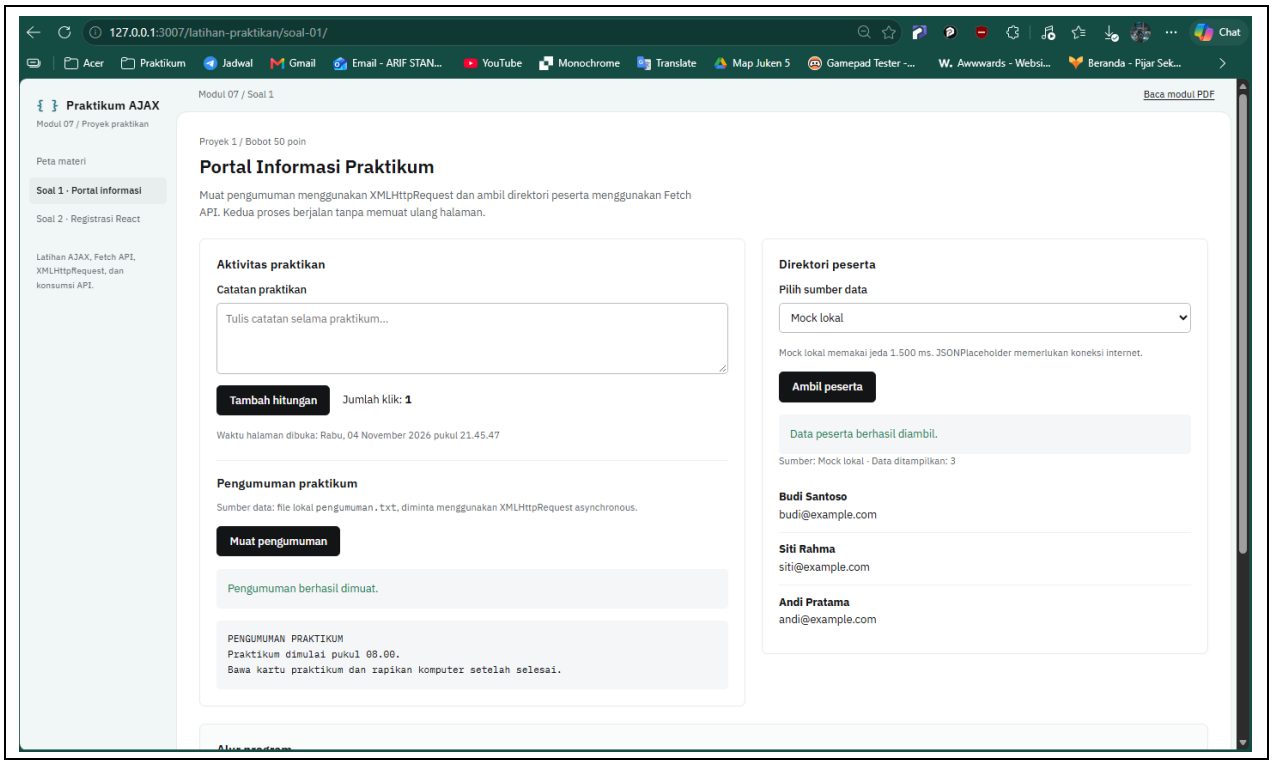
    const data = await response.json();
    if (!Array.isArray(data)) {
      throw new Error("Format response bukan array pengguna.");
    }

    const tigaPesertaPertama = data.slice(0, 3);
    infoPeserta.textContent = `Sumber: ${konfigurasi.nama} · Data ditampilkan:
${tigaPesertaPertama.length}`;

    if (tigaPesertaPertama.length === 0) {
      statusPeserta.dataset.state = "success";
      statusPeserta.textContent = "Request berhasil, tetapi tidak ada data
peserta.";
    } else {
      const fragmen = document.createDocumentFragment();
      tigaPesertaPertama.forEach((peserta) => {
```

```
        const item = document.createElement("li");
        const nama = document.createElement("strong");
        const email = document.createElement("div");
        nama.textContent = peserta.name ?? "Nama tidak tersedia";
        email.textContent = peserta.email ?? "Email tidak tersedia";
        item.append(nama, email);
        fragmen.append(item);
      });
      daftarPeserta.append(fragmen);
      statusPeserta.dataset.state = "success";
      statusPeserta.textContent = "Data peserta berhasil diambil.";
    }
  } catch (error) {
    statusPeserta.dataset.state = "error";
    if (pilihan === "eksternal") {
      statusPeserta.textContent = `Gagal mengambil API eksternal
(${error.message}). Periksa koneksi internet atau pilih Mock lokal secara
manual.`;
    } else {
      statusPeserta.textContent = `Gagal mengambil data dari Mock lokal:
${error.message}. Pastikan server praktikum berjalan.`;
    }
    infoPeserta.textContent = `Sumber: ${konfigurasi.nama} · Data ditampilkan:
0`;
  } finally {
    tombolPeserta.disabled = false;
  }
});
```

Screenshot:



Deskripsi Program:

Pada soal nomor 1 ini saya membuat program portal informasi praktikum berbasis HTML dan JavaScript yang digunakan untuk menampilkan pengumuman praktikum dan daftar peserta tanpa reload halaman. Program ini menerapkan konsep AJAX melalui dua metode, yaitu XMLHttpRequest untuk mengambil data pengumuman dari file pengumuman.txt dan Fetch API untuk mengambil data peserta dari API lokal maupun eksternal. Pengujian sudah dilakukan sesuai dengan modul, hasilnya berhasil semua. Berikut adalah penjelasan event pemicu, request yang dikirim, response yang diterima, dan elemen DOM yang berubah:

a. Event Pemicu

- Klik Muat pengumuman untuk mengambil file pengumuman.txt.
- Klik Ambil peserta untuk mengambil data dari Mock lokal atau JSONPlaceholder.
- Klik Tambah hitungan untuk menambah angka sebanyak satu.

b. Request yang Dikirim

- XMLHttpRequest: GET pengumuman.txt.
- Fetch lokal: GET /api/users?delay=1500.
- Fetch eksternal: GET https://jsonplaceholder.typicode.com/users.
- Tombol hitungan tidak mengirim request ke server.

c. Response yang Diterima

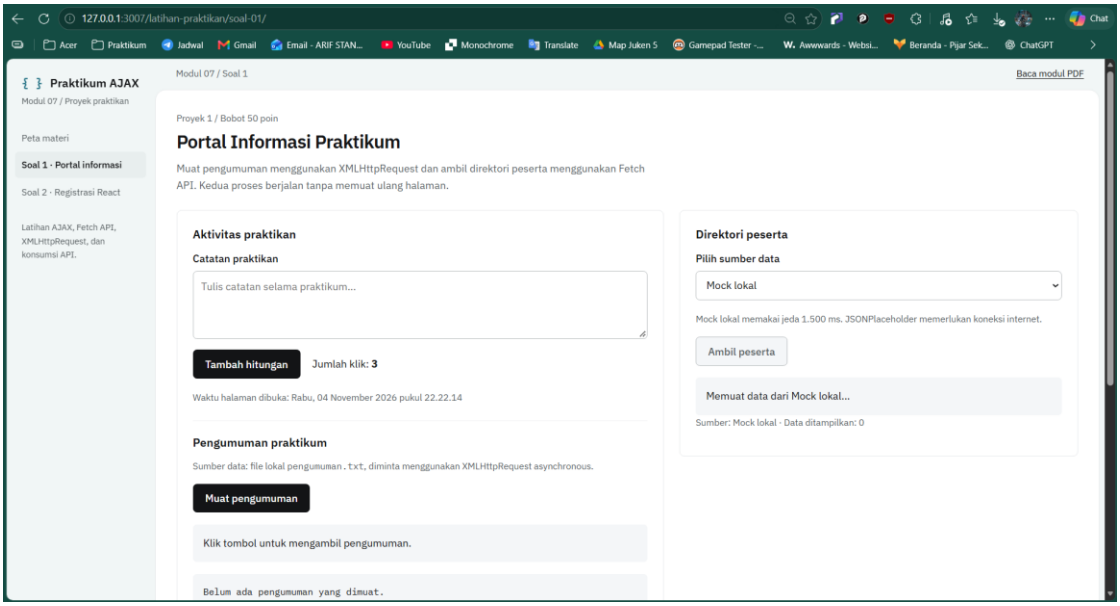
- XHR menerima teks pengumuman melalui responseText.
- Fetch menerima data peserta dalam format JSON, maksimal tiga data pertama.
- Jika request gagal, program menampilkan pesan kesalahan sesuai kondisi HTTP atau jaringan.

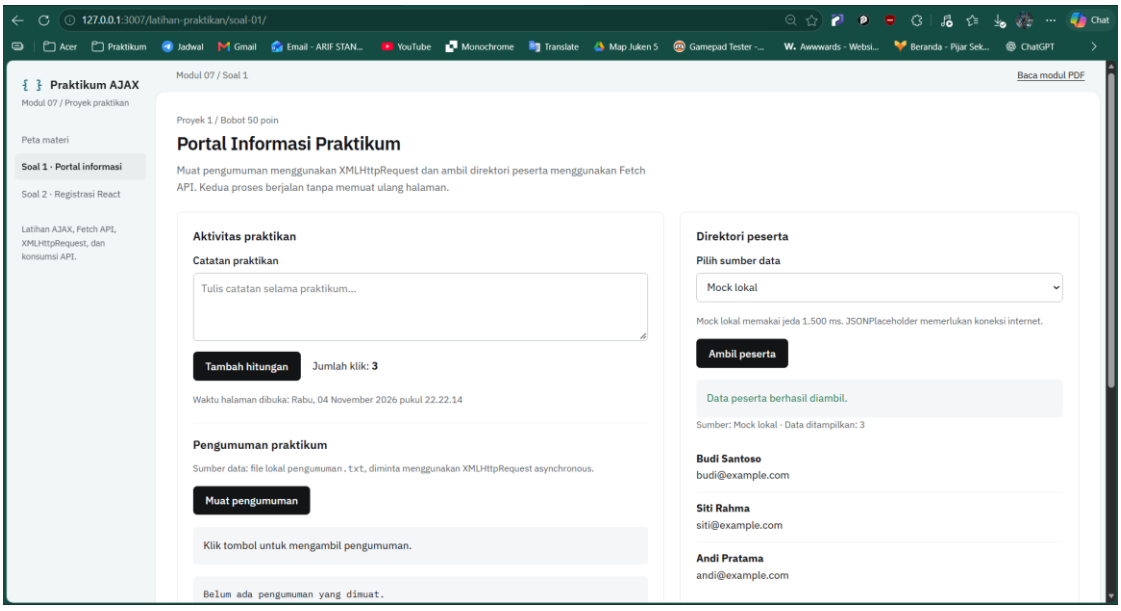
d. Elemen DOM yang Berubah

Elemen	Perubahan
#status-pengumuman	Status loading atau pesan hasil.
#isi-pengumuman	Isi pengumuman.
#status-peserta	Status pengambilan data.

#info-peserta	Sumber dan jumlah data.
#daftar-peserta	Nama dan email peserta.
#jumlah-hitungan	Angka hitungan bertambah.

e. Bukti Interaksi Hitungan Tetap Berjalan





Soal 02. Aplikasi registrasi peserta React

Kode:

index.html

```
<!doctype html>
<html lang="id">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Soal 2 | Registrasi Peserta React</title>
  <link rel="icon" href="data:,">
  <link rel="stylesheet" href="../../assets/style.css">
  <style>
    .form-actions { display: flex; align-items: center; gap: 12px; flex-wrap:
wrap; }
    .response-data { margin: 12px 0 0; padding: 14px; border-radius: 6px;
background: var(--raised); }
    .response-data p { margin: 4px 0; overflow-wrap: anywhere; }
    .response-data strong { display: inline-block; min-width: 112px; }
    .field-help { color: var(--muted); font-size: 14px; margin-top: -10px; }
    .section-divider { margin-top: 24px; padding-top: 20px; border-top: 1px
solid var(--rule); }
  </style>
</head>
<body>
  <a class="skip" href="#utama">Lewati navigasi</a>
  <aside class="sidebar">
    <a class="brand" href="../../index.html"><span class="brand-mark" aria-
hidden="true">{ }</span> Praktikum AJAX</a>
    <div class="scope">Modul 07 / Proyek praktikan</div>
    <nav aria-label="Navigasi proyek">
      <a href="../../index.html">Peta materi</a>
      <a href="../soal-01/">Soal 1 · Portal informasi</a>
      <a href="../soal-02/" aria-current="page">Soal 2 · Registrasi React</a>
    </nav>
    <p class="sidebar-footer">Latihan Axios GET/POST, state, effect, dan
validasi formulir.</p>
  </aside>

  <header class="topbar">
    <span>Modul 07 <span class="context">/ Soal 2</span></span>
    <a href="../../Modul_07_AJAX_Sederhana.pdf">Baca modul PDF</a>
  </header>

  <div class="mobile-nav">
    <label class="skip" for="pindah-proyek">Pilih proyek</label>
    <select id="pindah-proyek" aria-label="Pindah proyek"
onchange="location.href=this.value">
      <option value="../soal-01/">Soal 1 · Portal informasi</option>
      <option value="../soal-02/" selected>Soal 2 · Registrasi React</option>
    </select>
```

```
  </div>

  <main id="utama" class="sheet">
    <div class="file-warning" data-file-warning hidden>
      <strong>Jalankan server terlebih dahulu.</strong><br>
      Ekstrak seluruh ZIP, jalankan <code>JALANKAN.bat</code>, lalu buka halaman
melalui
      <a href="http://127.0.0.1:3007">http://127.0.0.1:3007</a>. Jangan membuka
file melalui <code>file://</code>.
    </div>
    <div class="page-meta">Proyek 2 / Bobot 50 poin</div>
    <h1>Aplikasi Registrasi Peserta React</h1>
    <p class="intro">Daftar peserta dimuat otomatis menggunakan Axios GET.
Formulir mengirim pendaftaran melalui Axios POST dan menampilkan response secara
terpisah.</p>
    <div id="root"></div>
    <div class="pager">
      <a href="../soal-01/">&larr; Kembali ke Soal 1</a>
      <a href="../../index.html">Peta materi &rarr;</a>
    </div>
  </main>

  <script src="../../assets/ui.js"></script>
  <script src="../../vendor/axios.min.js"></script>
  <script src="../../vendor/react.development.js"></script>
  <script src="../../vendor/react-dom.development.js"></script>
  <script src="../../vendor/babel.min.js"></script>
  <script type="text/babel" data-presets="react" src="app.jsx"></script>
</body>
</html>
```

app.jsx

```
const { useEffect, useState } = React;

const skenarioGet = {
  berhasil: {
    label: "Berhasil",
    url: "/api/users?delay=1000",
    timeout: 5000
  },
  kosong: {
    label: "Data kosong",
    url: "/api/users?mode=empty&delay=1000",
    timeout: 5000
  },
  error: {
    label: "Error HTTP",
    url: "/api/users?mode=error&delay=1000",
    timeout: 5000
  },
  timeout: {
    label: "Timeout",
```

```
    url: "/api/users?delay=2500",
    timeout: 700
  }
};

function AplikasiRegistrasi() {
  const [peserta, setPeserta] = useState([]);
  const [loading, setLoading] = useState(true);
  const [errorGet, setErrorGet] = useState("");
  const [skenario, setSkenario] = useState("berhasil");
  const [nomorMuatUlang, setNomorMuatUlang] = useState(0);

  const [nama, setNama] = useState("");
  const [email, setEmail] = useState("");
  const [sedangMengirim, setSedangMengirim] = useState(false);
  const [hasilPost, setHasilPost] = useState(null);

  // GET berjalan otomatis saat awal dan setiap skenario / permintaan ulang
berubah.
  useEffect(() => {
    const controller = new AbortController();
    let effectAktif = true;
    const konfigurasi = skenarioGet[skenario];

    async function ambilPeserta() {
      setLoading(true);
      setErrorGet("");
      setPeserta([]);

      try {
        const response = await axios.get(konfigurasi.url, {
          timeout: konfigurasi.timeout,
          signal: controller.signal
        });
        if (effectAktif) {
          setPeserta(Array.isArray(response.data) ? response.data : []);
        }
      } catch (error) {
        if (!effectAktif || axios.isCancel(error)) return;

        if (error.code === "ECONNABORTED" || error.code === "ETIMEDOUT") {
          setErrorGet(`Request timeout. Batas tunggu ${konfigurasi.timeout} ms
telah tercapai.`);
        } else if (error.response) {
          const pesanServer = error.response.data?.message || "Server menolak
request.";
          setErrorGet(`HTTP ${error.response.status}: ${pesanServer}`);
        } else {
          setErrorGet("Tidak dapat terhubung ke server. Pastikan server
praktikum masih berjalan.");
        }
      } finally {
```

```
        if (effectAktif) setLoading(false);
      }
    }

    ambilPeserta();
    return () => {
      effectAktif = false;
      controller.abort();
    };
  }, [skenario, nomorMuatUlang]);

  async function kirimPendaftaran(event) {
    event.preventDefault();
    setHasilPost(null);

    const namaBersih = nama.trim();
    const emailBersih = email.trim();
    const polaEmail = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

    if (!namaBersih) {
      setHasilPost({
        jenis: "error",
        pesan: "Nama wajib diisi dan tidak boleh hanya berisi spasi."
      });
      return;
    }

    if (!polaEmail.test(emailBersih)) {
      setHasilPost({
        jenis: "error",
        pesan: "Format email tidak valid. Contoh: nama@example.com."
      });
      return;
    }

    const objekData = { name: namaBersih, email: emailBersih };
    setSedangMengirim(true);

    try {
      const response = await axios.post("/api/users", objekData);
      setHasilPost({
        jenis: "success",
        status: response.status,
        data: response.data
      });
    } catch (error) {
      if (error.response) {
        setHasilPost({
          jenis: "error",
          pesan: `HTTP ${error.response.status}: ${error.response.data?.message
|| "Pendaftaran gagal."}`
        });
```

```
      } else {
        setHasilPost({
          jenis: "error",
          pesan: "Pendaftaran gagal karena server tidak dapat dijangkau. Periksa
koneksi ke server lokal."
        });
      }
    } finally {
      setSedangMengirim(false);
    }
  }

  const sedangMemuatPeserta = loading;
  const adaPeserta = !loading && !errorGet && peserta.length > 0;
  const dataKosong = !loading && !errorGet && peserta.length === 0;

  return (
    <div className="workspace">
      <section className="practice" aria-label="Daftar peserta" aria-
busy={loading}>
        <h2>Daftar peserta</h2>
        <label htmlFor="skenario-get">Skenario GET</label>
        <select
          id="skenario-get"
          value={skenario}
          onChange={(event) => setSkenario(event.target.value)}
        >
          <option value="berhasil">Berhasil</option>
          <option value="kosong">Data kosong</option>
          <option value="error">Error HTTP</option>
          <option value="timeout">Timeout</option>
        </select>
        <div className="form-actions">
          <button
            type="button"
            disabled={loading}
            onClick={() => setNomorMuatUlang((nomor) => nomor + 1)}
          >
            Muat ulang
          </button>
          <span className="field-help">Timeout mengikuti skenario yang
dipilih.</span>
        </div>
        <p
          className="result"
          role="status"
          aria-live="polite"
          data-state={errorGet ? "error" : (adaPeserta ? "success" : "")}
        >
          {sedangMemuatPeserta
            ? "Memuat daftar peserta..."
            : errorGet
```

```
              ? errorGet
              : dataKosong
                ? "Request berhasil, tetapi data peserta kosong."
                : `Berhasil memuat ${peserta.length} peserta.`}
        </p>
        {adaPeserta && (
          <ul className="users">
            {peserta.map((user) => (
              <li key={user.id}>
                <strong>{user.name}</strong>
                <div>{user.email}</div>
              </li>
            ))}
          </ul>
        )}
        <p className="field-help">Response GET hanya ditampilkan pada bagian
ini. Hasil POST tidak mengubah daftar peserta.</p>
      </section>

      <section className="practice" aria-label="Formulir pendaftaran">
        <h2>Formulir pendaftaran</h2>
        <form onSubmit={kirimPendaftaran} noValidate>
          <label htmlFor="nama-peserta">Nama</label>
          <input
            id="nama-peserta"
            type="text"
            maxLength="80"
            value={nama}
            onChange={(event) => setNama(event.target.value)}
            placeholder="Masukkan nama lengkap"
          />

          <label htmlFor="email-peserta">Email</label>
          <input
            id="email-peserta"
            type="email"
            maxLength="120"
            value={email}
            onChange={(event) => setEmail(event.target.value)}
            placeholder="nama@example.com"
          />

          <div className="form-actions">
            <button type="submit" disabled={sedangMengirim}>
              {sedangMengirim ? "Mengirim..." : "Kirim pendaftaran"}
            </button>
            {sedangMengirim && <span className="field-help">Mengirim data ke API
lokal...</span>}
          </div>
        </form>

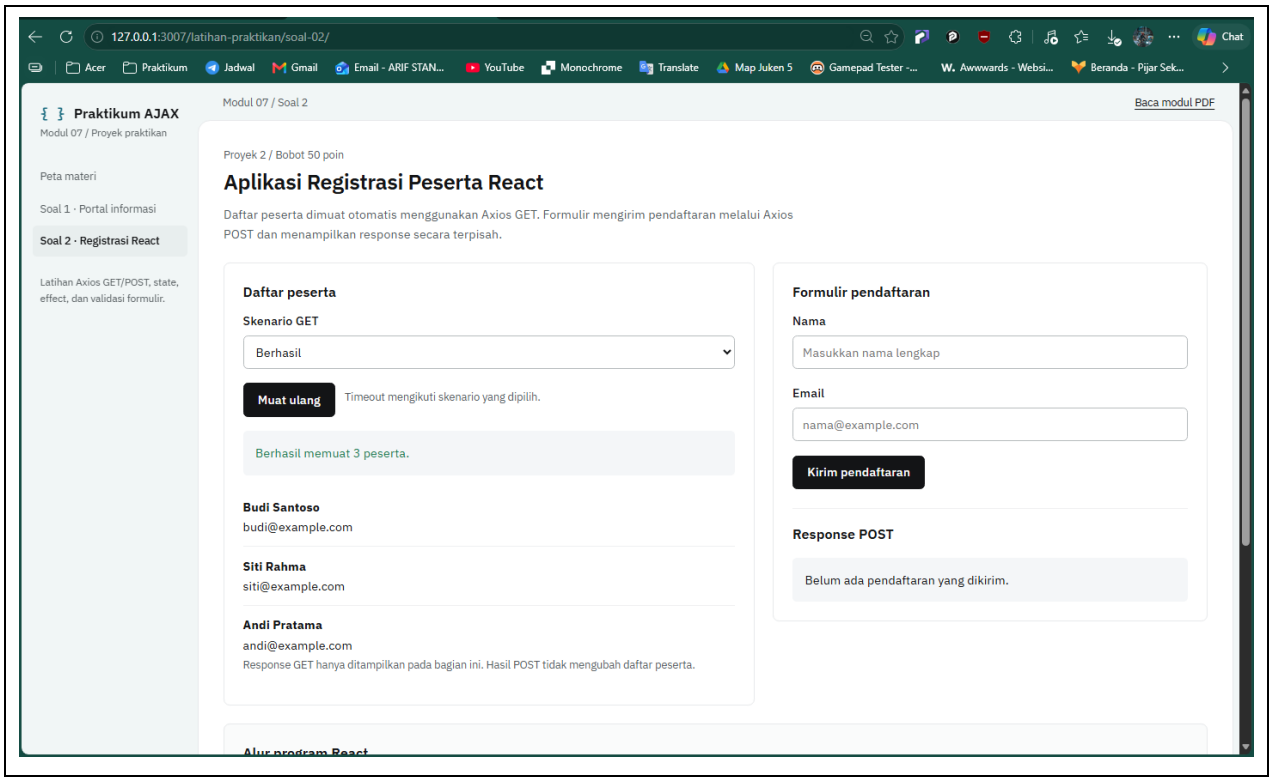
        <div className="section-divider">
```

```
          <h2>Response POST</h2>
          {!hasilPost && (
            <p className="result" role="status">Belum ada pendaftaran yang
dikirim.</p>
          )}
          {hasilPost?.jenis === "error" && (
            <p className="result" role="alert" data-
state="error">{hasilPost.pesan}</p>
          )}
          {hasilPost?.jenis === "success" && (
            <div className="result" role="status" data-state="success">
              <strong>Simulasi berhasil, data tidak disimpan.</strong>
              <div className="response-data">
                <p><strong>Status HTTP:</strong> {hasilPost.status}</p>
                <p><strong>ID:</strong> {hasilPost.data.id}</p>
                <p><strong>Nama:</strong> {hasilPost.data.name}</p>
                <p><strong>Email:</strong> {hasilPost.data.email}</p>
                <p><strong>Simulated:</strong>
{String(hasilPost.data.simulated)}</p>
              </div>
            </div>
          )}
        </div>
      </section>

      <section className="explanation" style={{ gridColumn: "1 / -1" }}>
        <h2>Alur program React</h2>
        <ol>
          <li><code>useState</code> menyimpan daftar peserta, status loading dan
error, skenario GET, input formulir, serta status pengiriman.</li>
          <li><code>useEffect</code> menjalankan Axios GET saat komponen pertama
kali tampil dan saat skenario berubah.</li>
          <li><code>AbortController</code> dan penanda effect aktif mencegah
response lama mengubah tampilan setelah skenario berubah atau komponen
dilepas.</li>
          <li>Axios GET membaca daftar dari <code>response.data</code>. Tombol
Muat ulang menaikkan nomor request agar effect berjalan kembali.</li>
          <li>Formulir memanggil <code>preventDefault()</code>, memvalidasi nama
dan email, lalu mengirim objek <code>name</code> dan <code>email</code> melalui
Axios POST.</li>
          <li>Response POST ditampilkan terpisah. Server mengembalikan simulasi
HTTP 201 dan tidak menambahkan pendaftar ke daftar GET.</li>
        </ol>
      </section>
    </div>
  );
}

ReactDOM.createRoot(document.querySelector("#root")).render(<AplikasiRegistrasi
/>);
```

Screenshot:



Deskripsi Program:

Pada soal nomor 2 ini saya membuat aplikasi registrasi peserta berbasis React yang digunakan untuk menampilkan daftar peserta dan mengirimkan data pendaftaran melalui API. Program menggunakan Axios untuk melakukan request GET dan POST, serta useState dan useEffect untuk mengelola data dan proses pengambilan informasi. Daftar peserta dimuat secara otomatis saat halaman dibuka dan dapat dimuat ulang melalui tombol Muat ulang. Program juga menyediakan pilihan skenario pengujian, yaitu Berhasil, Data kosong, Error HTTP, dan Timeout. Selain itu, tersedia formulir pendaftaran yang berisi nama dan email dengan validasi input sebelum data dikirim ke server. Program menampilkan status loading, hasil request, dan pesan kesalahan. Data pendaftaran yang dikirim menghasilkan response simulasi HTTP 201 dan tidak disimpan ke server. Pengujian sudah dilakukan dan hasilnya berhasil semua.